

Customer Churn Prediction — Spec-Driven Development Plan

Stack: XGBoost (modeling) + MySQL (data & feature storage) **Methodology:** Spec-Driven Development (Requirements → Design → Tasks, in that order, with each phase gated on sign-off of the previous spec)

1. Problem Statement & Goals

Problem: The business needs to identify customers likely to churn within a defined future window so that retention teams can intervene before they leave.

Primary goal: Produce a scored, ranked list of active customers with a calibrated churn probability, refreshed on a schedule, backed by a reproducible MySQL → feature store → XGBoost pipeline.

Out of scope (v1): Causal "why did they churn" explanations beyond feature importance/SHAP, real-time (sub-second) scoring, multi-product churn (assume single product/subscription).

2. Requirements Specification

2.1 Functional Requirements

ID	Requirement
FR1	System ingests customer, subscription, usage, billing, and support data into MySQL on a defined cadence.
FR2	System defines and materializes a churn label per customer per observation window.
FR3	System computes a versioned feature set per customer, stored in MySQL.
FR4	System trains an XGBoost binary classifier on historical features + labels.
FR5	System evaluates the model against held-out data before promoting it.
FR6	System scores all active customers and writes probabilities + risk tier to MySQL.
FR7	System exposes model version, training date, and metrics for auditability.
FR8	System supports retraining on a schedule or on-demand trigger.

2.2 Non-Functional Requirements

ID	Requirement
----	-------------

NFR1	Batch scoring for ~1M customers completes in a defined SLA window (e.g., < 30 min) — adjust to actual scale.
------	--

NFR2	Feature computation is deterministic and reproducible from raw MySQL tables (no hidden state).
------	--

NFR3	Data quality checks run before training/scoring; pipeline fails loudly rather than silently producing bad scores.
------	---

NFR4	Class imbalance (churners are typically a minority class) is explicitly handled and documented.
------	---

NFR5	Model artifacts and MySQL schema are version-controlled together.
------	---

NFR6	PII fields are identified and access-controlled separately from feature/model tables.
------	---

2.3 Success Metrics / Acceptance Criteria

- **Primary metric:** PR-AUC (preferred over ROC-AUC given class imbalance) $\geq$ agreed baseline on time-based holdout.
- **Secondary metrics:** Recall@top-decile (how many actual churners are captured in the top 10% riskiest customers), F1 at chosen operating threshold, calibration error (Brier score).
- **Business acceptance:** Retention team can act on the top-N list and precision at that N is high enough to justify outreach cost — define this threshold with stakeholders before Phase 3.

3. Data Specification

3.1 Data Sources (typical for a churn problem — adjust to your actual system)

- **Customer master data:** demographics, signup date, plan/tier
- **Subscription/billing data:** plan changes, payment history, invoices, failed payments
- **Usage/engagement data:** logins, feature usage, session frequency
- **Support data:** tickets, complaints, NPS/CSAT scores
- **Churn events:** cancellation date, downgrade date, reason code if captured

3.2 MySQL Schema Design (raw layer)

```
sql
```

```

CREATE TABLE customers (
    customer_id      BIGINT UNSIGNED PRIMARY KEY,
    signup_date      DATE NOT NULL,
    plan_tier        VARCHAR(50),
    region           VARCHAR(100),
    acquisition_channel VARCHAR(100),
    created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_signup_date (signup_date)
);

CREATE TABLE subscriptions (
    subscription_id   BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    customer_id       BIGINT UNSIGNED NOT NULL,
    plan_name         VARCHAR(100),
    monthly_price     DECIMAL(10,2),
    status            ENUM('active','cancelled','paused') NOT NULL,
    start_date        DATE NOT NULL,
    end_date          DATE NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    INDEX idx_customer_status (customer_id, status)
);

CREATE TABLE billing_events (
    event_id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    customer_id       BIGINT UNSIGNED NOT NULL,
    event_type        ENUM('payment_success','payment_failed','refund','upgrade','downgrade'),
    amount            DECIMAL(10,2),
    event_date        DATETIME NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    INDEX idx_customer_date (customer_id, event_date)
);

CREATE TABLE usage_events (
    event_id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    customer_id       BIGINT UNSIGNED NOT NULL,
    event_type        VARCHAR(100),
    event_date        DATETIME NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    INDEX idx_customer_date (customer_id, event_date)
);

```

```

CREATE TABLE support_tickets (
    ticket_id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    customer_id        BIGINT UNSIGNED NOT NULL,
    opened_at          DATETIME NOT NULL,
    resolved_at        DATETIME NULL,
    csat_score          TINYINT NULL,
    category            VARCHAR(100),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    INDEX idx_customer_opened (customer_id, opened_at)
);

CREATE TABLE churn_labels (
    customer_id        BIGINT UNSIGNED NOT NULL,
    observation_date    DATE NOT NULL,    -- the "as of" date features are computed from
    label_window_end    DATE NOT NULL,    -- end of the future window checked for churn
    churned             TINYINT(1) NOT NULL,
    PRIMARY KEY (customer_id, observation_date),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

```

3.3 Data Quality Rules

- No `usage_events` or `billing_events` dated after `observation_date` may leak into features for that row (strict point-in-time correctness).
- `churn_labels.churned = 1` requires a verified cancellation/non-renewal, not just inactivity, unless inactivity-based churn is explicitly the defined label.
- Reject training runs if null rate on any core feature column exceeds a defined threshold (e.g., 5%) without an explicit imputation rule.

4. Feature Engineering Specification

4.1 Feature Categories

- **Tenure:** days since signup, plan age, number of plan changes
- **Engagement:** login frequency (7/30/90-day windows), trend (declining vs. stable usage), days since last activity
- **Billing/financial:** payment failures in last N days, monthly spend, discount usage, downgrade history
- **Support:** ticket count, average CSAT, unresolved ticket flag
- **Static:** plan tier, region, acquisition channel

4.2 Feature Store Table (materialized in MySQL)

sql

```
CREATE TABLE feature_store (  
  customer_id          BIGINT UNSIGNED NOT NULL,  
  observation_date      DATE NOT NULL,  
  feature_set_version   VARCHAR(20) NOT NULL,  
  days_since_signup     INT,  
  logins_last_30d       INT,  
  logins_last_90d       INT,  
  usage_trend_ratio     DECIMAL(6,3),  
  payment_failures_90d  INT,  
  monthly_spend         DECIMAL(10,2),  
  open_tickets          INT,  
  avg_csat_90d          DECIMAL(4,2),  
  days_since_last_activity INT,  
  plan_tier             VARCHAR(50),  
  PRIMARY KEY (customer_id, observation_date, feature_set_version)  
);
```

4.3 Label Definition

Define explicitly and document in the spec, e.g.: *"Churned = subscription status becomes 'cancelled' within 30 days after observation_date, and was 'active' at observation_date."* Every feature row must join to exactly one label row on `((customer_id, observation_date))`.

5. Model Specification

5.1 Why XGBoost

Handles tabular, mixed-type data well; native handling of missing values; built-in class-imbalance controls (`scale_pos_weight`); fast to iterate on; strong track record on churn-style problems.

5.2 Training Pipeline

1. Pull `feature_store` JOIN `churn_labels` for a given `feature_set_version` from MySQL into a training dataframe.
2. Time-based train/validation/test split (not random) — train on older observation dates, validate/test on more recent ones, to simulate real deployment.
3. Encode categoricals (target/one-hot or native XGBoost categorical support).
4. Train XGBoost with early stopping on validation set.

5. Log model, metrics, feature importances, and hyperparameters to a model registry table.

5.3 Hyperparameter Search Space (starting point)

```
yaml

objective: binary:logistic
eval_metric: aucpr
n_estimators: [200, 400, 800]
max_depth: [3, 4, 5, 6]
learning_rate: [0.01, 0.05, 0.1]
subsample: [0.7, 0.8, 1.0]
colsample_bytree: [0.7, 0.8, 1.0]
scale_pos_weight: computed as (negatives / positives) in training set
min_child_weight: [1, 3, 5]
early_stopping_rounds: 30
```

Use Bayesian search (e.g., Optuna) or randomized search with cross-validation on the time-based training window; never tune on the final test set.

5.4 Handling Class Imbalance

- Set `scale_pos_weight` from the actual training-set ratio.
- Evaluate with PR-AUC, not accuracy.
- Consider threshold tuning post-hoc based on business cost of false positives (wasted outreach) vs. false negatives (missed churner).

6. Evaluation Specification

6.1 Metrics

PR-AUC, ROC-AUC (secondary), precision/recall at chosen threshold, recall captured in top-decile, calibration (Brier score / reliability curve).

6.2 Validation Strategy

Time-based split: e.g., train on observation dates up to month M, validate on M+1, test on M+2 (a true forward-looking holdout). Never shuffle observation dates randomly — this leaks future information.

6.3 Acceptance Thresholds

Define concrete numeric gates before training starts (fill in with stakeholders), e.g.:

- PR-AUC $\geq$ X on holdout
- Precision $\geq$ Y at the outreach-list size the retention team can act on

- No feature with importance > 50% (guards against label leakage via a single dominant feature)

7. System Architecture

MySQL (raw tables)

- ETL / feature computation job (SQL or Python)
- feature_store table (MySQL)
- training job (Python + XGBoost) → model_registry table + serialized model file
- scoring job (loads latest approved model, reads feature_store, writes predictions)
- model_predictions table (MySQL) → consumed by BI dashboard / retention tooling

Component responsibilities:

- **ETL job:** point-in-time correct feature computation, data quality checks, writes to `feature_store`.
- **Training job:** trains, validates against acceptance thresholds, registers model if it passes; rejects if it doesn't.
- **Scoring job:** loads the currently-approved model, scores all active customers, writes results with model version for traceability.
- **Model registry table:** tracks model version, training date, metrics, and approval status.

sql

```

CREATE TABLE model_registry (
  model_id          VARCHAR(50) PRIMARY KEY,
  trained_at        DATETIME NOT NULL,
  feature_set_version VARCHAR(20) NOT NULL,
  pr_auc            DECIMAL(6,4),
  precision_at_k     DECIMAL(6,4),
  approved          TINYINT(1) DEFAULT 0,
  artifact_path      VARCHAR(255)
);

CREATE TABLE model_predictions (
  customer_id       BIGINT UNSIGNED NOT NULL,
  model_id          VARCHAR(50) NOT NULL,
  scored_at         DATETIME NOT NULL,
  churn_probability DECIMAL(6,5),
  risk_tier         ENUM('low', 'medium', 'high') NOT NULL,
  PRIMARY KEY (customer_id, model_id)
);

```

8. Deployment & Serving Specification

- Batch scoring job runs on a schedule (daily/weekly, per business need), reads latest `feature_store` rows, scores with the current `approved = 1` model.
- Model promotion is a manual or automated gate: a new model only flips `approved = 1` after passing Section 6.3 thresholds against the previous production model as a baseline (challenger vs. champion).
- Rollback plan: keep the prior approved model's artifact and registry row so scoring can revert if the new model underperforms in production.

9. Monitoring & Retraining Specification

- **Data drift:** monitor feature distributions (e.g., mean/variance of key features) week over week.
- **Performance drift:** once true churn outcomes are known for a scored cohort, compute realized precision/recall and compare to training-time metrics.
- **Retraining trigger:** scheduled (e.g., monthly) or drift-triggered if realized precision drops below an agreed floor.
- **Alerting:** pipeline failures, data quality rule violations, and metric degradation should all alert the owning team.

10. Task Breakdown (Phased Plan)

Phase	Deliverable	Key Tasks
0 — Setup	Environment & repo scaffolding	Provision MySQL schema, set up Python env (xgboost, pandas, sqlalchemy/pymysql), version control
1 — Data Foundation	Raw tables populated & validated	Build ETL for customers/subscriptions/billing/usage/support tables; write data quality checks
2 — Labeling	<code>churn_labels</code> table	Formalize churn definition with stakeholders; implement point-in-time label generation
3 — Feature Engineering	<code>feature_store</code> table, v1 features	Implement feature SQL/Python jobs; validate no leakage; document each feature
4 — Model Development	Trained XGBoost model + evaluation report	Time-based split; hyperparameter search; evaluate against Section 6 thresholds
5 — Registry & Scoring	<code>model_registry</code> , <code>model_predictions</code> tables live	Implement training→registry write; implement scoring job
6 — Validation with Business	Sign-off on acceptance thresholds	Review top-N list with retention team; adjust threshold/risk tiers
7 — Deployment	Scheduled pipeline in production	Orchestration (cron/Airflow/etc.), alerting, rollback tested
8 — Monitoring & Iteration	Drift dashboard, retraining cadence	Track realized performance; define retraining trigger; plan v2 features

11. Risks & Assumptions

- **Assumption:** A reliable churn/cancellation event exists in source data; if churn is inferred from inactivity only, the label definition needs extra scrutiny.
- **Risk:** Label leakage — any feature computed using data at or after the churn event will inflate offline metrics and fail in production. Point-in-time joins (Section 3.3/4.3) mitigate this.
- **Risk:** Class imbalance may make accuracy misleading; the plan mitigates this by standardizing on PR-AUC.

- **Risk:** Feature/label definitions drifting between training and scoring if `feature_set_version` isn't enforced consistently — mitigated by versioning in the schema.
-

12. Open Questions for Stakeholders

1. What exactly defines "churn" for this business (cancellation vs. non-renewal vs. inactivity)?
2. What is the prediction horizon (churn within 30/60/90 days)?
3. What outreach capacity exists (how many customers can retention realistically contact), which determines the operating threshold?
4. What's the required refresh cadence for scores?